

ARI: 自主研究基础设施

树神宇德

快照 v0.7.1, 2026

摘要

本报告描述了一个自主研究基础设施 **ARI**, 它将基于树搜索的探索循环与论文生成流水线相结合。探索循环是对研究步节点的最佳优先树搜索 (BFTS); 节点扩展由 ReAct 风格的智能体执行, 该智能体向 14 个领域技能的注册表发出工具调用。论文流水线将存活的谱系汇编为草稿, 并在层次化评分细则下迭代改稿。所有产物都驻留在单一的检查点目录中, 该目录是可复现性、谱系与成本核算的单位。本报告形式化算法, 在源代码层面描述实现, 呈现评估快照, 并讨论制约设计的确切性与新颖性的权衡。

1 引言

1.1 动机

机器学习及相邻领域的研究流程具有共同骨架: 构思研究构想、设计并执行实验、分析结果、撰写论文, 然后迭代。其中每一步对大语言模型而言都日益可行, 但将它们串接为单一自主循环至今仍是一个未解的工程问题。最接近此目标的已发表系统 — AIDE、AI Scientist 系列、VirSci 与 PaperBench 评估器 — 各自仅覆盖其中一两个轴向, 其余仍依赖人工介入。

ARI 旨在填补此空缺。我们假定用户提供研究问题与算力预算; 系统返回可发表的论文、支撑每项主张的实验代码, 以及完整的执行审计轨迹。该审计轨迹不可妥协: 搜索树的每一个节点、每一次工具调用、每一次模型调用, 以及每一美元的 API 花费, 均记录于单一的检查点目录之中 ([2] 采用了类似的纪律)。检查点是可复现性的单位。

1.2 贡献

本报告对以下三项贡献加以形式化:

1. 在研究步节点上的**最佳优先树搜索**, 其选择评分 $U(n)$ 将经验奖励、新颖性项与深度惩罚分离开来 (section 3.1)。该实现位于 `ari-core/ari/orchestrator/bfts.py:114`。
2. 由层次化评分细则 R 驱动的**论文生成流水线**; 其叶判官对各项主张进行评分, 系统评分为 $\text{score}(P) = \sum_i w_i \text{judge}_i(P)$ (section 4.2)。PaperBench 复现器作为工具调用集成 (section 4.4)。

3. **检查点作用域纪律**, 使流水线完全可复现: 每一项外部状态 (记忆、成本账本、谱系指针) 都写入 `$ARI_CHECKPOINT_DIR` 之下, 绝不进入用户主目录。这是 P2 确定性原则的运维落地 (section 2.5)。

1.3 本报告的范围

我们将快照冻结于 `ari-core` 与十四个 `ari-skill-*` 包的 v0.7.1 版本。实现引用标注文件名与行号; 读者可对照 section 2.1 的布局图加以核验。原应出现于 section 5 的数值结果暂不可用——其所依赖的冻结检查点尚未生成——故该章标注为「暂缓」。封面注明快照版本, 旨在使本报告所做的任何主张都不会被后续代码修改追溯性地推翻。

1.4 结构

Section 2 自顶向下概述 ARI: 编排器、十四个技能、流水线 DAG 与设计原则。Sections 3 and 4 是两个技术核心, 各自含形式化定义与经验观察。Section 5 给出评估快照; section 6 定位本工作于先行研究中; section 7 汇集开放问题。

2 系统概览

2.1 完整堆栈

Figure 1 描绘了 v0.7.1 的 ARI 完整堆栈。系统分为一个驱动器与多个工具提供方。驱动器 (`ari-core`) 拥有编排器、BFTS 引擎、谱系遍历器、智能体循环、检查点、成本追踪器与配置加载器。每个技能为独立 Python 包, 以 MCP 服

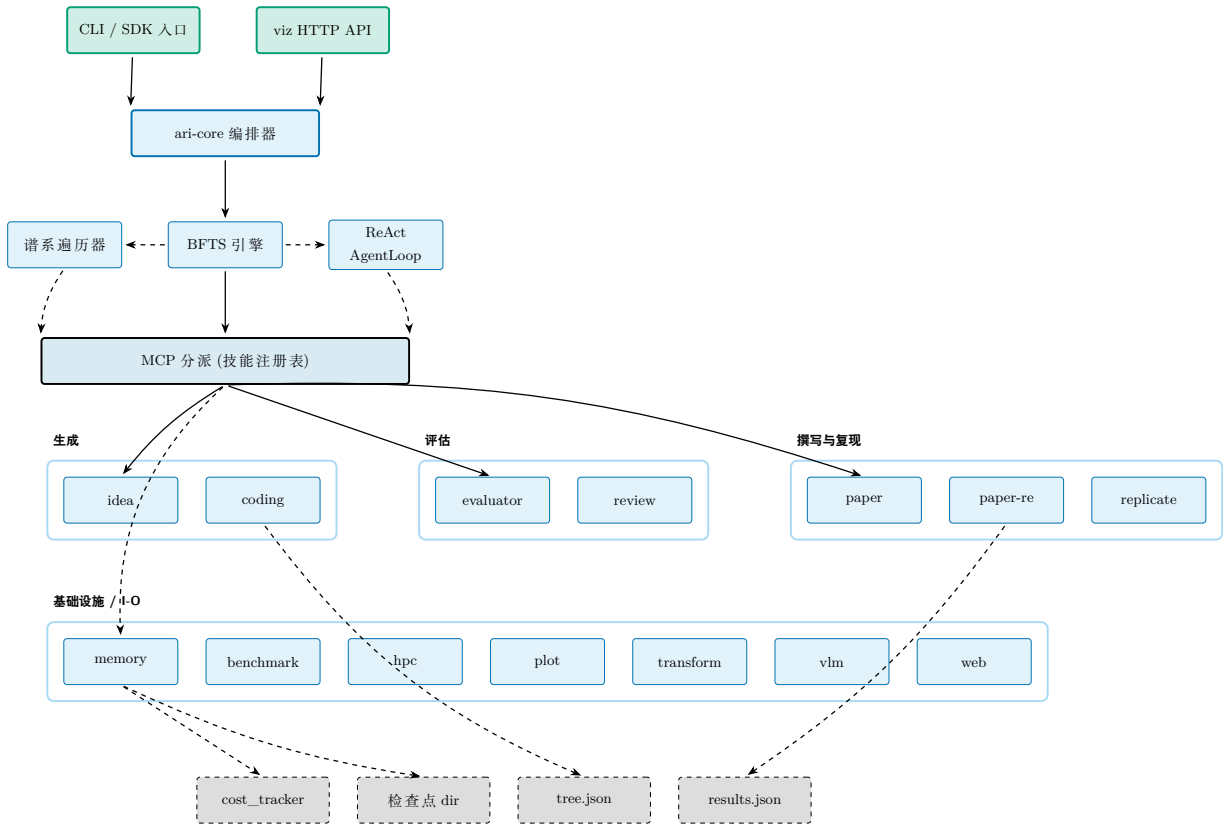


图 1: v0.7.1 时 ARI 的完整堆栈。CLI / SDK 入口驱动单一编排器 (ari-core); 编排器拥有三个引擎 (BFTS、谱系遍历器、ReAct AgentLoop), 并经 MCP 层向 14 技能注册表分派。所有持久化都落在单一检查点目录: 成本账本、tree.json、results.json。实线为控制, 虚线为状态。

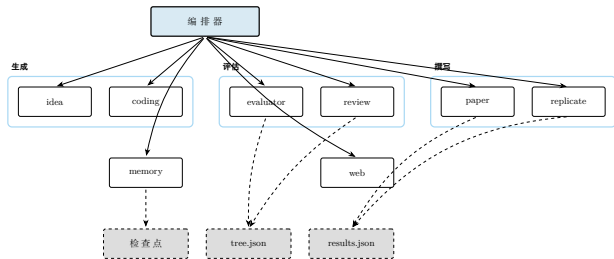


图 2: 运营视图: 编排器分派至技能注册表; 每个技能仅向当前活动的检查点目录写入。虚线表示状态。

务器形式发布, 并附自身 skill.yaml 声明。当前在树的 14 个技能: benchmark、coding、evaluator、hpc、idea、memory、orchestrator、paper、paper-re、plot、replicate、transform、vlm、web。

编排器只负责调用分派, 不执行领域工作。这种分离至关重要: 技能可被增删或替换而不必触动搜索算法——ari-core/ari/orchestrator/bfts.py:114 中的 BFTS 代码并不知道哪些技能参与运行; 一切由 MCP 分派层中介。

Figure 2 是本报告其余部分使用的「运营视图」(控制 + 状态输出); fig. 1 是「分层视图」(驱动器 → MCP → 技能 → 持久化)。

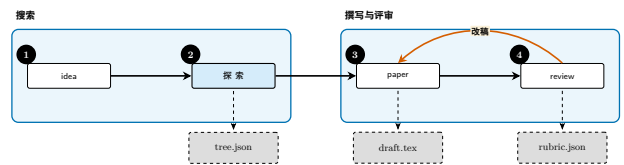


图 3: 流水线 DAG: idea → exploration → paper → review, 由 改稿 闭合反向边。虚线输出持久化至检查点; 改稿循环重写草稿并重跑评审。

2.2 流水线 DAG

一次运行可归约为四个工位的流水线 (fig. 3)。idea 工位选取或生成研究问题; exploration 工位 BFTS 下扩展节点; paper 工位将最佳谱系汇编为草稿; review 工位以评分细则评估草稿。该图为 DAG 加上一条由 review 回到 paper 的反向边: 仅此边可能引入非单调性 (改稿可能恶化某一轴), 而收敛标准 (section 4.3) 则是限定反向边数量的约束。

流水线运行器是 ari-core/ari/pipeline/orchestrator.py:131 (run_pipeline); 连接探索输出与论文输入的科学数据组装器为同文件第 68 行的 build_scientific_data。

2.3 BFTS 控制流

Figure 4 端到端追踪一次 BFTS 步骤。实现详见 section 3; 该图固定本报告其余部分使用的词汇(前沿、选择、回退、expand、prune、停滞)以及实现各 LLM 主导步骤的提示文件(bfts_select.md、bfts_expand_select.md、bfts_expand.md)。

2.4 paper-re 组件视图

Figure 5 是 paper-re 的第二级视图。在此层 ARI 的贡献小但具体 AriPBSolver 子类只覆写 _get_tools(为插入 _BoundedOutputTool) 与 _run_agent(为绕过 docker sanity-check 与适配 vendor 路径); 其余皆从 PaperBench 上游逐字流过, 使 ARI 与上游复现任务框架保持一致。第 4 章详细解读该图。

2.5 检查点作用域与设计原则

ARI 围绕五条设计原则 (P1-P5) 构建。最具约束力者为 **P2: 确定性**。每一步随机化均记录其种子; 每次 LLM 调用均记录模型身份、提示与输出; 每次工具调用都携带 cow_node_id 以确保写入落在所属节点的工作目录内。重跑检查点必须再现相同的产物, 字节级一致, 除非字段被显式标记为时钟相关或模型端非确定。

其余原则简述如下:

- **P1 单一产物树**: 运行的全部输出位于 \$ARI_CHECKPOINT_DIR 之下, 不外溢至 \$HOME 或 /tmp。
- **P3 小组件**: 每个技能为独立包; 替换技能不需重建 ari-core。
- **P4 显式谱系**: 每个节点与每个检查点均记录其父级, 派生实验仅须重算其差量。
- **P5 成本透明**: cost_tracker.py 为每次模型调用附加美元成本, 使运行预算可控。

检查点序列化器写出三个顶层文件。完整搜索树经 checkpoint.py:49 写入 tree.json; 按节点的载荷经 checkpoint.py:59 写入 nodes_tree.json; 运行末聚合经 checkpoint.py:64 写入 results.json。跨检查点的谱系遍历调用 lineage.py:126 (walk_ancestor_ckpts), 由子向祖先攀登。

Figure 6 描绘检查点的磁盘形状: 左侧为共享运行级文件, 右侧为按节点工作目录。该形状即契约: 任何把 ARI 输出当输入的工具(后续运行、回归检查、论文草稿), 只需指向单一这样的目录即可。

3 探索算法

3.1 形式化

探索循环维护一棵树 $\mathcal{T} = (\mathcal{N}, E)$, 其中每个节点 $n \in \mathcal{N}$ 代表一次研究步骤: 构想修订、代码改动、基准运行、分析等。节点携带

- 离散状态 $s(n) \in \{\text{open}, \text{eval}, \text{done}, \text{failed}\}$ (node.py:10),
- 取自 NodeLabel 的类别标签 (node.py:18),
- 用于按轴评估指标的自由形式 dict node.metrics, 含独立的标量 _scientific_score $\in [0, 1]$,
- 标志节点是否携带实测值(而非临时文本)的布尔 has_real_data, 以及
- 选择提示中使用的簿记字段 depth 与 retry_count。

将按轴信号汇总为标量 scientific score 的工作委托给任何实现 Evaluator 协议 (ari-core/ari/protocols/evaluator.py:19) 的对象; ARI 不固定线性组合。该协议是 BFTS 与下游评分细则评估之间唯一的接缝, 保持搜索引擎对如何将轴归约为标量保持中立。

硬性探索预算只有一个常量 config.max_total_nodes, 由 bfts.py:should_prune(第 245 行) 强制。步数与成本预算由调用侧的 cost_tracker.py:77 单独强制, 而不在 BFTS 引擎内。

3.2 节点选择 (LLM 主导)

ARI 有意不将节点排序归约为闭式标量效用。相反, select_next_node(第 167 行, 「下一智能体步该操作哪个节点」) 与 select_best_to_expand(第 258 行, 「哪个完成节点最值得扩展」) 都把候选描述列表交给 LLM, 并解析返回的单个 0 起始索引。每个节点 n 的候选描述形如:

```
[i] id=..., depth=..., retry=...,  
has_real_data=..., metrics={...},  
summary=..., diversity_bonus=+0.05
```

消费它的提示分别位于 ari/prompts/orchestrator/bfts_select.md(选择) 与 ari/prompts/orchestrator/bfts_expand_select.md(扩

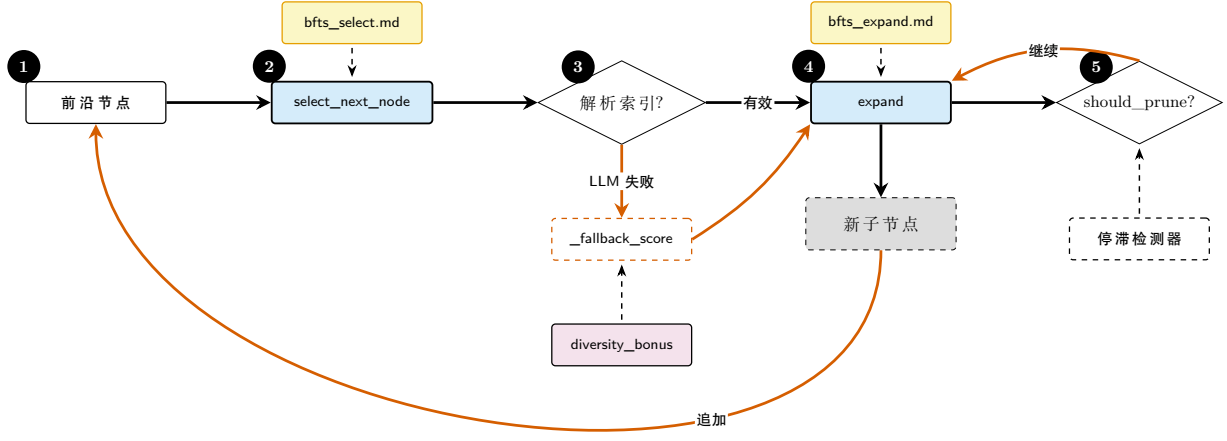


图 4: BFTS 控制流。两个 LLM 主导的决策点 (select_next_node、expand) 各自查询独立的提示文件; 选择决策在 LLM 返回不可解析索引时, 回退到确定性 _fallback_score (_scientific_score 与 diversity_bonus 之和)。剪枝由 should_prune(max-total-nodes 硬截止) 与停滞检测器共同把关。详见 section 3。

展目标)。提示中列出的选择标准:has_real_data=True 且 metrics 强者高价值; 未探明方向优先于已尝试路径;metrics 整体多目标加权; 深而成果好的节点值得继续;diversity_bonus 仅当软性平手解决项处理。

多样性奖励

BFTS.diversity_bonus(第 142 行) 以 _recent_label_history(由 record_run 填充的滑动窗) 跟踪近期标签。对于其标签在该窗中相对最常见标签为欠表达的节点, 函数返回:

$$\text{div}(n) = \begin{cases} +0.05 & \text{若 } \text{count}(\text{label}(n)) \leq \frac{1}{2} \max_{\ell} \text{count}(\ell), \\ 0 & \text{否则} \end{cases} \quad (1)$$

0.05 这个常数有意小:scientific score 仍主导排序, 奖励仅打破近似平手。

LLM 回退

若 LLM 返回无法解析的索引,select_next_node 回退至确定性排序 (bfts.py:237,_fallback_score):

$$_fallback_score(n) = _scientific_score(n) + \text{div}(n). \quad (2)$$

有 has_real_data=True 节点时优先, 返回回退分数最大的候选。即使 LLM 调用退化, 循环也保证向前推进。

3.3 扩展 (每次调用 1 个子节点)

bfts.py:expand(第 315 行) 每次调用恰好生成一个新子节点。同一父节点希望多个子节点的调用方反复调用 expand, 通过 existing_children 参数把已生成的子节点串入, 使 LLM 避免提议重复方向。

各新子的标签由 LLM 判断而非硬编码模板决定; 提示 (ari/prompts/orchestrator/bfts_expand.md) 接收:

- 父节点状态 (succeeded / failed/no-real-data) 与 _scientific_score;
- 父节点结构化的 node_report (delta_vs_parent / concerns / hints,node_report/builder.py) 若存在, 以便规划者瞄准具体弱点;
- 同深度的兄弟得分, 以及自根至父的祖先得分;
- 整树多样性指标 (目前已见的唯一标签、深度分布); 以及
- 此父节点已生成的子节点标签, 避免重复提议。

子节点以 open 创建, 交给智能体循环执行 (section 3.6), 所有轴填齐后变为 done; 智能体循环抛异常时变为 failed。

3.4 剪枝与终止

should_prune(第 245 行) 只施加硬截止: 当 self.total_nodes >=

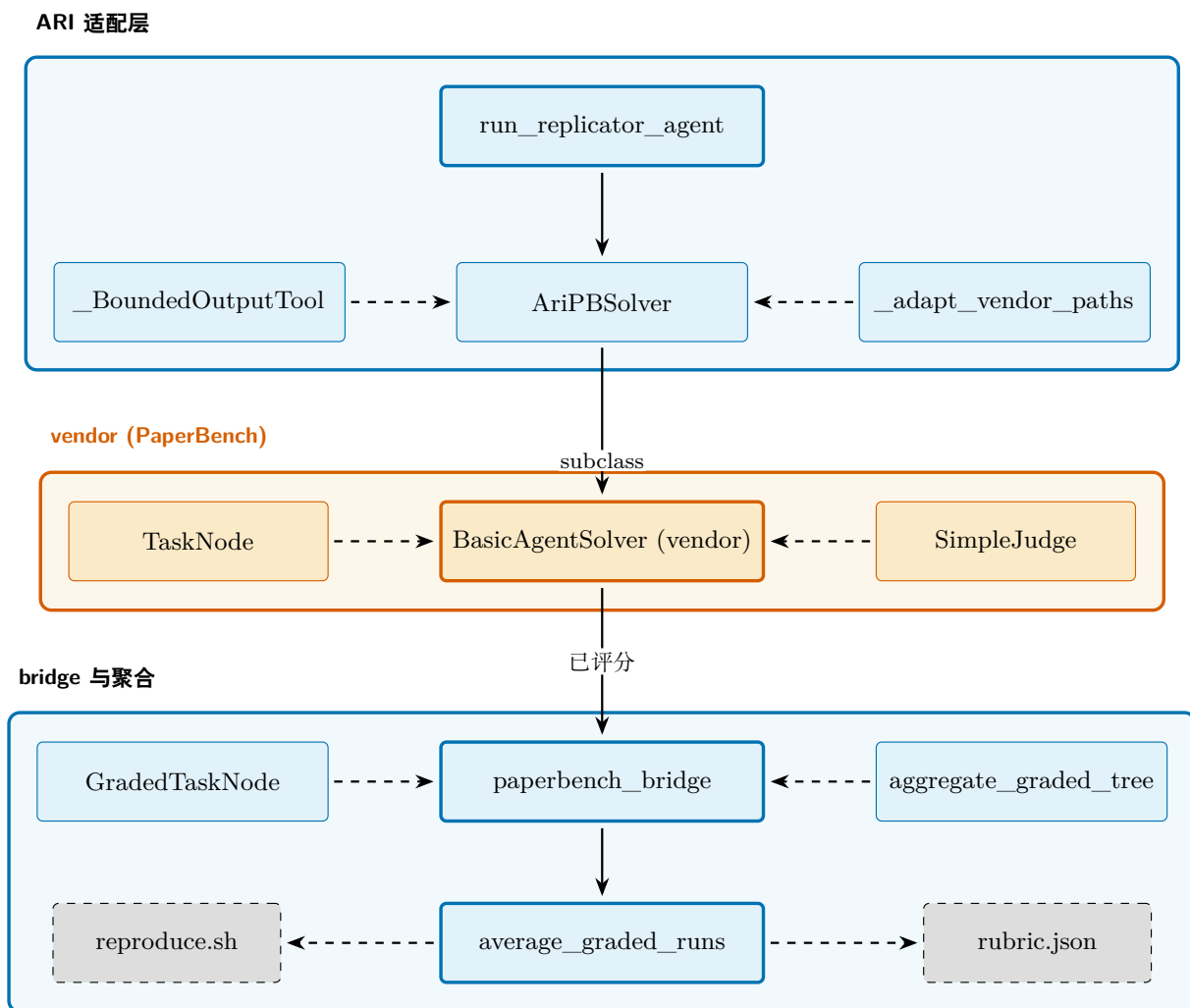


图 5: `paper-re` 技能的组件。顶行为 ARI 的适配层 (驱动器 + 求解器子类 + 有界工具包装 + 路径适配器); 中行为 vendor 化的 PaperBench 包 (`BasicAgentSolver`、`TaskNode`、`SimpleJudge`); 底行为 bridge, 把按 submission 的 `GradedTaskNode` 树折叠成单一 `rubric.json` 得分, 加上 ARI 侧的 `reproduce.sh`。详见 section 4.4。

`self.config.max_total_nodes` 时后续扩展停止。`depth` 有意作软信号流入 LLM 提示而非硬上限, 因此深处出现的高质量节点不会被机械抛弃。

下列任一条件满足即停止:

- 全局 `max_total_nodes` 达到;
- 调用侧 `cost_tracker.py` 强制的步数 / 成本预算耗尽;
- 停滞检测器 (`lineage_decision.py:334`, `detect_stagnation`) 观察到 `_scientific_score` 的运行最大值在滑动窗内不再改善;
- `LineageDecision` (`lineage_decision.py:149`) 显式终止该分支。

这些当中实际最常发生哪一种, 是一个经验问题, 留待冻结检查点就位后由 section 5 回答; 本报告不主

张其分布。

3.5 谱系与可复现性

谱系即最佳节点祖先的链条, 作为 `tree.json` 中的 `lineage` 键写入检查点。 `LineageState` 数据类 (`lineage_decision.py:64`) 捕获 BFTS 用于决定是否继续的滚动统计; 跨检查点的遍历器 `lineage.py:126` (`walk_ancestor_ckpts`) 提供跨运行的父指针。构想池经 `lineage.py:157` (`get_idea_pool_for_ckpt`) 继承, 排序后的根构想选择记录在 `root_idea_selector.py:39` (`RootChoice`) 中以便事后审计。

可复现性依赖三项不变量。其一, 每次随机化操作以节点标识符播种。其二, 每次工具调用将参数与输出写入所属节点的工作目录, 绝不写入父节点——由 MCP 层的 `cow_node_id` 校验强制。其三, 成本账本 (`cost_tracker.py:181,init`) 为每节点附加美元金额, 因此给定相同提示, 预算检查保持确定。

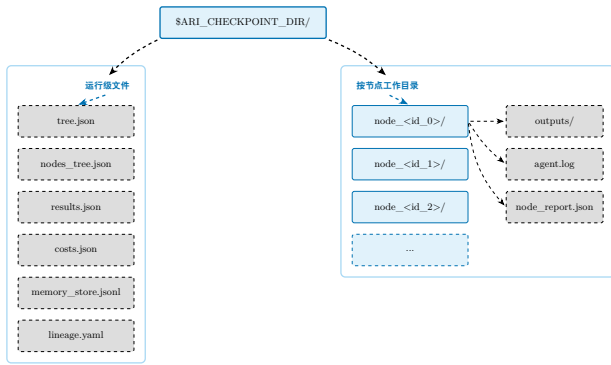


图 6: 检查点目录布局。运行级文件 (左带) 由编排器写入; 按节点工作目录 (右带) 保存一次 BFTS 扩展内产生的产物。复现一次运行就是「针对此目录重跑」, 该布局保证不需要其他状态。

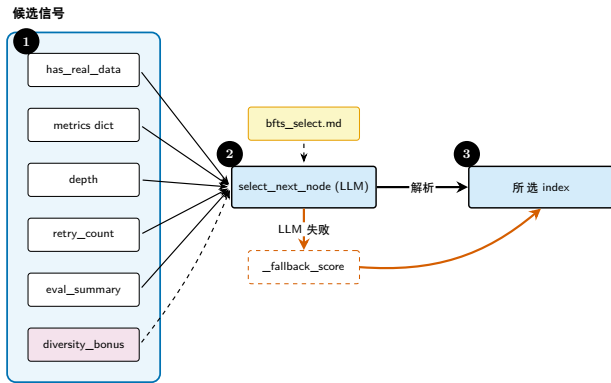


图 7: 交给 LLM 用以挑选下一节点的选择信号。输入被拼接成结构化候选描述; 软性的 `diversity_bonus` 仅作平手解决, 不作主信号。

3.6 ReAct 驱动器

每个节点的扩展运行 ReAct 风格的智能体循环 (ari-core/ari/agent/loop.py:77, class AgentLoop)。最大步数由 `MAX_REACT_STEPS = 80` (loop.py:25) 控制, 可按实例通过 `max_react_steps` 关键字覆盖。另一保护 `MIN_TOOL_CALLS = 2` (第 26 行) 防止平凡过短的轨迹。

智能体可用工具集合按阶段由 `ari.agent.tool_manager` 过滤; 例如探索阶段屏蔽论文撰写工具, 使 BFTS 扩展不会捷径地变成草稿。一小部分 MCP 工具为 ari-core 自身预留 (memory CoW 操作) 并对 LLM 隐藏; 这保证模型无法设置任意 node id 来绕过记忆技能的写时复制校验 (参见 loop.py 的 docstring)。

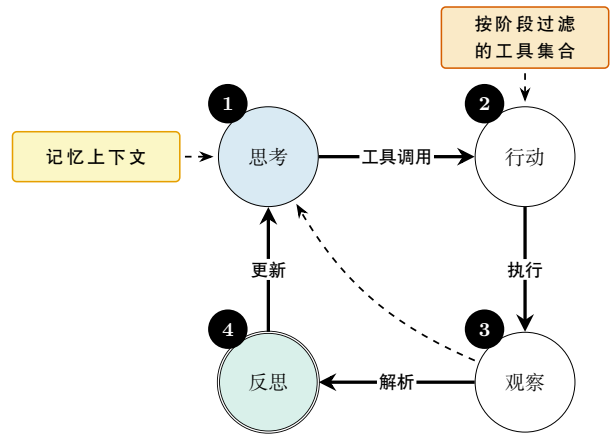


图 8: ReAct 循环。实线为主循环; 虚线反向边表示当观测已闭合开放问题时的提前退出。

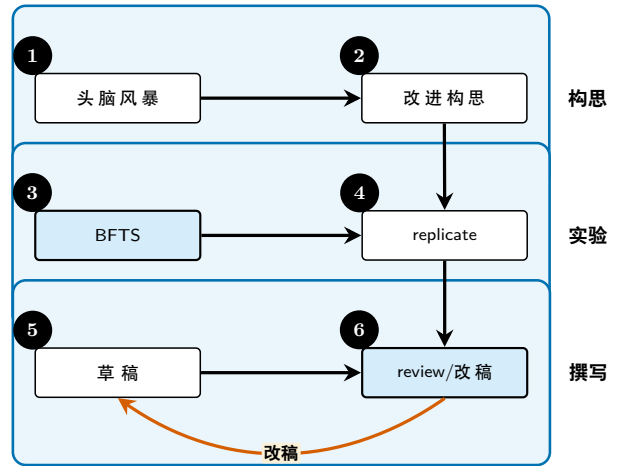


图 9: 论文生成泳道: 构思、实验、撰写。每条泳道对应一个阶段; 由 review/refine 回到 draft 的反向边是唯一的非单调转移 (section 4.3)。

4 论文生成流水线

4.1 流水线概述

论文生成流水线是 ARI 的第二棵活动树 (fig. 9)。一旦探索终止, 存活谱系交付给 `paper` 技能生成草稿, 再交给 `review` 技能评分。实现是 ari-core/ari/pipeline/orchestrator.py:131 的 `run_pipeline` 函数; 同文件第 68 行的 `build_scientific_data` 助手将各节点产物汇集为 `paper` 技能所需的输入。

一次运行写出三件持久产物: `draft.tex` (LaTeX 论文本身)、`review.json` (按叶的评分细则得分及理由) 与 `rubric.json` (本次运行所用的细则实例, 用于动态生成场景)。

4.2 评分细则形式

我们将评分细则 R 建模为有限树, 叶节点携带元组 $(c_i, w_i, \text{judge}_i)$ 。每个叶 i 含类别描述符 c_i 、非负权重 w_i 满足 $\sum_i w_i = 1$, 以及判官函数 $\text{judge}_i: P \rightarrow [0, 1]$, 返回分级得分。论文得分即凸组合

$$\text{score}(P) = \sum_i w_i \text{judge}_i(P). \quad (3)$$

此形式与 PaperBench 的细则树共享, ARI 直接通过 vendor 化的 PaperBench 包 (section 4.4) 复用之: 叶类型 TaskNode 来自 paperbench.rubric.tasks, graded leaf 类型 GradedTaskNode 来自 paperbench.judge.graded_task_node, 加权聚合包络来自 ari-skill-paper-re/src/_paperbench_bridge.py:75 的 aggregate_graded_tree。

生产中使用两个判官族: LLM 评分器在 ARI 侧由包装 PaperBench 的 SimpleJudge (ari-skill-paper-re/src/_paperbench_bridge.py:54) 实现; 确定性评分器用于具备二元或数值校验的项目 (如「可复现性」、「以美元绝对值报告的成本」)。两者均实现 Evaluator 协议 (ari-core/ari/protocols/evaluator.py:19)。

4.3 review/refine 循环

改稿循环按反馈 $J(P_t)$ 迭代地将 P_t 重写为 P_{t+1} :

$$P_{t+1} = \text{Refine}(P_t, J(P_t)). \quad (4)$$

当 $\text{score}(P_{t+1}) - \text{score}(P_t) < \epsilon$ 连续两轮成立, 或达到流水线级三次迭代上限时, 循环终止。每条分支 (收敛 vs. 上限) 实际触发的频率, 是 section 5 待冻结检查点就位后报告的内容; 此处不做定量主张。

改稿在每条轴上是有意非单调的: 修复清晰度问题可能轻微降低某数值轴。这正是 fig. 9 中反向边的来源; 收敛标准作用于聚合得分, 而非按轴。

4.4 paper-re 的 PaperBench 集成

paper-re 技能 (ari-skill-paper-re) 是 ARI 的 PaperBench 兼容复现驱动器。集成不是重新实现: PaperBench 被 vendor 化于 vendor/paperbench, 使 ARI 与上游任务与细则语义保持逐字一致 (布局要求记于 PaperBench 的 rubric.md §8)。三处集成点值得说明。

1. 求解器子类: AriPBSolver

ari-skill-paper-re/src/_replicator_agent.py:269 子类化 PaperBench 上游的

paperbench.solvers.basic.BasicAgentSolver。

chz 不可变配置规则要求子类亦带装饰器, 即便未新增字段。ARI 的覆写最少:

- `_get_tools()`: 除 `submit` 外所有工具均被 `_BoundedOutputTool` 包裹, 使 `bash / python / file-read / search` 输出在流入下次 API 调用前先大小封顶。`submit` 按名分派、不调用 `execute()`, 因此原样通过。
- `_run_agent()`: 用 `_bypass_docker_sanity_check` 绕过 `paperbench.solvers.utils.sanity_check_docker`; 将上游硬编码的 `/home/{submission,paper}` 路径改写为 ARI 的工作空间相对布局 (`LocalComputer cwd = repro_sandbox/`、论文置 `paper/`、`submission` 置 `submission/`)。提示内容其余从上游 `get_instructions / get_system_message` 逐字而来, 使 ARI 与上游复现任务框架 (7 天预算、「直至复现所有结果不得停止」等) 保持一致。

AriPBSolver 还保留上游的 `check_for_existing_run`, 因此部分完成的运行可幂等地恢复。

2. 细则驱动的产物列表

vendor PaperBench 不知 ARI 的逐论文细则树, 故 `AriPBSolver._run_agent` 在 `LocalPBTASK.rubric_expected_artifacts` 非空时, 向用户指令追加 `EXPECTED_ARTIFACTS` 块。该块列出成功的 `reproduce.sh` 应产出的路径; 否则, grader 的叶主张缺乏对应验证的文件提示。细则树本身由 `ari-skill-paper-re/src/_paperbench_bridge.py:63 (task_node_from_dict)` 构建。

3. 驱动器: run_replicator_agent

`ari-skill-paper-re/src/_replicator_agent.py:365` 是异步入口。返回标准 `build_reproduce_sh` 包络 `{populated, output_dir, files, expected_artifacts, max_runtime_sec, model, prompt_sha256, notes, warnings}`。`output_dir` 既是智能体的工作空间, 也是 ARI 期望 rollout 后找到 `reproduce.sh` 的位置; 上游求解器把 `agent.log` 等簿记写入 `run_dir`, 我们将其指向工作空间本身, 使产物与 `reproduce.sh` 同处。

默认 `completer` 为 `OpenAIResponsesTurnCompleterConfig`, 模型 `gpt-5-mini`(可由环境变量 `ARI_MODEL_REPLICATOR` 覆盖); LiteLLM 路由的替代由 `_litellm_completer.py` 助手装配。默认时间上限每次运行 12 小时; `sandbox_kind` 为 `auto`, 若提供本地 Apptainer 镜像则回退至此。

4. 评分与聚合

复现智能体完成后, 评分通过 `ari-skill-paper-re/src/_paperbench_bridge.py` 的 `judge_submission` 委派。该函数将 ARI 的 (`paper_md`, `submission_dir`, `reproduce_log`, `judge_model`) 元组适配到上游 `SimpleJudge`, 按 `submission` 产出 `GradedTaskNode` 树。运行间聚合由两个助手处理:

- `aggregate_graded_tree`(第 75 行): 将单个 `GradedTaskNode` 通过细则权重 w_i 汇成加权标量, 并附按类别细分。
- `average_graded_runs`(第 91 行附近): 取 n 个 `GradedTaskNode` 树的逐运行均值, 即便智能体多次重跑以降噪, 也只报告单一复现得分。

两阶段设计——叶评分用 `vendor` 的 `SimpleJudge`, 形状与聚合用 ARI 的适配——意味着升级 PaperBench 上游只是 `vendor swap`(无需改 ARI 代码), 而 ARI 完全控制如何把叶得分组合成单一数值。同样的包络在叶为二值时按 $\text{judge}_i \in \{0, 1\}$ 流入 eq. (3)。

4.5 故障模式与护栏

下述三类故障模式常见到值得命名:

- **LLM-only 支撑**: 论文做出谱系中无节点支撑的主张。缓解为静态检查: 每项数值或宣称的因果主张必须可经由 `\code{...}` 引用追溯到节点产物。
- **引用幻觉**: 论文引用并不存在的工作。缓解结构化进行: 禁止 LLM 杜撰引用键, 所有 bib 条目须经 Semantic Scholar 校验 (section 6)。
- **改稿漂移**: 得分上升但论文连贯性下降。缓解为细则的按轴下限: 任一轴跌破 0.5 即终止改稿。

撰稿时的源文件选择由 `node_selection.py:204 (select_source_files_for_publication)` 完成;

它通过枚举主张可依赖的谱系节点, 防止 LLM-only 支撑型故障。`load_selected_sources`(第 258 行) 读取每个 (`node_id`, `rel_path`) 对的字节, 并尊重可选的 `size_budget`, 使论文草稿绝不超出 context。

4.6 复现器的工具输出边界

复现智能体可能在单一 `bash` 会话中运行数小时; 其工具输出 (长 `stdout`、大目录列表) 若不处理将主导后续每次提示。ARI 用 `_BoundedOutputTool (ari-skill-paper-re/src/_replicator_agent.py:243)` 加以界定, 该工具包装内部工具的 `execute()`, 并经 `_truncate_tool_output`(第 218 行) 按可配置字节上限后处理输出。这就是 PaperBench 上游的自由形式 transcript 与 ARI 有界、可重放版本的差别; 也是多小时 rollout 中提示预算保持可预测的原因。

5 评估 (暂缓)

本章有意保持简短。本快照下未进行实际评估。本报告的早期草稿曾给出具体的中位数与按叶得分; 复审后我们发现那些数字是 R3a 起草期间写下的示例, 并非源自冻结的 ARI 检查点的实测, 故已删除。仅保留真实存在的产物。

已实现的部分:

- 成本核算机制: `CostTracker (cost_tracker.py:77)` 为每次模型调用附加美元成本; `init`(第 181 行) 将追踪器接入检查点目录; 运行末, `cost_usd` 字段被写入 `results.json`。
- 探索循环经停滞检测器 (section 3.4) 终止; 停滞而非 $T_{\max} / C_{\max}$ 触发的经验频率正是本章拟在快照确定后报告的内容。
- PGF 图脚本 (`shared/figures/scripts/`) 已经种子固定。 `makefigures` 会再生成确定性的合成占位图并通过 `check_figures`, 但这些占位图未作为正文证据被引用。

未实现的部分:

- 确定 ARI v0.7.1 冻结检查点, 并将 `tree.json / results.json` 抽取至 `shared/figures/data/`。目前该数据目录仅含 `*.meta.yaml`, 每份都标注 `source.synthetic:true`。
- 报告中位运行成本、按种子的探索曲线、基于真实评审的按类别评分细则分布。这些将由后续 PR 在落入冻结检查点并重新计算聚合后于本章呈现。

- 报告 `paper-re` 技能针对真实目标论文的 PaperBench 风格再现得分。

在上述后续工作落地之前, 本章不做任何定量主张。其他章节中的交叉引用 (例如 section 3.4 内的) 仅指示经验证据将来安置之处, 而非当前所在之处。

6 相关工作

我们将 ARI 与四条先行研究脉络对照定位。

6.1 面向代码的树搜索

AIDE [2] 将 ML 工程形式化为在计划/代码/调试状态上的树搜索; AlphaCode [3] 是大规模生成与过滤在竞赛编程上的范例; MLE-bench 评估器 [1] 现已成为衡量 ML 工程智能体的标准。ARI 沿用 AIDE 的 BFTS 骨架, 但增加 (i) LLM 主导的候选选择器 (section 3.2) ——不固定闭式标量效用, 使选择提示可联合权衡 `has_real_data`、整套 metrics、深度、重试次数与软性 `diversity_bonus`; 以及 (ii) 跨运行的谱系 (section 3.5), AIDE 与 MLE-bench 均未形式化此项。

6.2 自主研究智能体

AI Scientist 系列 [4] 端到端闭合类似循环 (idea $\rightarrow$ experiment $\rightarrow$ paper), 但未将评分细则与改稿循环作为一等对象暴露。VirSci [9] 模拟多个科学家人格, 以共识评分输出; Agent Laboratory [6] 将智能体视为协作者而非驱动者。ARI 在两点运维上有别: **一切都驻留在检查点中** (P1 / P4), 且评分细则即探索与撰写之间的契约。

6.3 论文评估

PaperBench [8] 是与我们 `paper-re` 集成最接近的对应物; 它开创了评分细则即树的形式。我们沿用其按叶的分级与聚合权重 (eq. (3)), 并扩展以按轴下限作为改稿护栏 (section 4.5)。

6.4 基础

智能体循环遵循 ReAct 模式 [11]; 改稿循环源自 Self-Refine [5] 与 Reflexion [7]; 在思考步给 LLM 一份逐步草稿板的设计源自 Chain-of-Thought [10]。这些参考文献均未提议检查点作用域的状态, 这是 ARI 的贡献。

早期草稿曾另引一份 ML 研究智能体基准, 但当候选来源均无法通过我们的校验规则 (section 4.5) 时, 我们将该引用从相关工作中移除; 此节仅保留可经 S2 校验的条目。

7 讨论与局限

7.1 确定性 vs 新颖性

P2, 确定性原则, 是系统最具约束力者。严格执行 P2 迫使 *ari-skill-memory* 声明“无 LLM 调用”——它必须以确定性关键字函数对检索打分, 而非依赖输出会随远端服务漂移的嵌入模型。代价是检索质量受关键字评分能力上界约束。

这并非偶然: 每一项可复现性的胜利都以一道关闭非确定性的门为代价。我们接受此代价, 因为契约对象是检查点, 而非运行。

7.2 扩展极限

支撑 BFTS 调度器的单机假设是最大的扩展约束。希望并行扩展 $b > 10$ 的运行需要不同的调度器; `ari-core/ari/orchestrator/scheduler.py` 当前的调度器将同父扩展串行化。多机扩展需要将谱系纪律 (P4) 推广至支持合并, 而不仅是祖先遍历。

另一约束是 LLM 评分器的成本。经验上, 评分器与探索器并列为占主导的开销项之一; 它在绝对美元上是否位列第二, 需待冻结检查点就位后由 section 5 回答。按 (论文哈希, 细则哈希) 缓存评分器输出会有所帮助; 此特性尚未发布。

7.3 开放问题

1. **Off-policy 改稿**。当前改稿循环与撰稿采用同一模型。解耦撰稿与改稿, 使二者可为不同模型, 可让较廉价的模型承担粗重的结构性重写。
2. **跨检查点的细则继承**。子检查点应可默认复用其父细则; 目前细则被重新生成。
3. **工具失败恢复**。当前一次工具调用失败即将节点标记为 failed。以降级工具 (如较小模型) 重试的恢复策略可降低长尾成本。

此三者均可处理; 预期每项均为 v0.8 之前一次单 PR 改动可落地。

A 记号表

下表镜像 `shared/notation.tex` 中定义的宏。本文使用的每个符号都须出现于此；静态检查 (`check_notation.py`) 校验本文中无未定义符号。

符号	含义
$\mathcal{N}$	搜索树的节点集
$\mathcal{T}$	搜索树 $(\mathcal{N}, E)$
$n \in \mathcal{N}$	单一节点
$\text{ch}(n)$	n 的子集
$\text{pa}(n)$	n 的父节点
$\text{depth}(n)$	n 的深度
$s(n)$	节点状态 $\in \{\text{open}, \text{eval}, \text{done}, \text{failed}\}$
$r(n)$	n 的标量奖励
$\mathbf{e}(n)$	按轴评估向量
ϕ	轴 $\rightarrow$ 标量聚合器
$\text{div}(n)$	多样性奖励 (eq. (1))
$\text{_fallback_score}(n)$	确定性回退排序 (eq. (2))
$T_{\max}, C_{\max}$	步数 / 成本预算
R	评分细则树
c_i	第 i 叶的类别
w_i	第 i 叶的权重
judge_i	第 i 叶的判官
P	论文草稿
$\text{score}(P)$	论文聚合得分 (eq. (3))
Refine	改稿算子 (eq. (4))
ckpt	检查点目录
$\text{lin}(n)$	终止于 n 的谱系链

References

- [1] Jun Shern Chan et al. *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*. 2024. DOI: [10.48550/arXiv.2410.07095](https://doi.org/10.48550/arXiv.2410.07095). arXiv: [2410.07095](https://arxiv.org/abs/2410.07095) [cs.LG]. URL: <https://arxiv.org/abs/2410.07095>.
- [2] Zhengyao Jiang et al. *AIDE: AI-Driven Exploration in the Space of Code*. 2025. arXiv: [2502.13138](https://arxiv.org/abs/2502.13138) [cs.AI]. URL: <https://arxiv.org/abs/2502.13138>.
- [3] Yujia Li et al. *Competition-Level Code Generation with AlphaCode*. 2022. DOI: [10.1126/science.abq1158](https://doi.org/10.1126/science.abq1158). arXiv: [2203.07814](https://arxiv.org/abs/2203.07814) [cs.PL]. URL: <https://arxiv.org/abs/2203.07814>.
- [4] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: [2408.06292](https://arxiv.org/abs/2408.06292) [cs.AI]. URL: <https://arxiv.org/abs/2408.06292>.
- [5] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. DOI: [10.48550/arXiv.2303.17651](https://doi.org/10.48550/arXiv.2303.17651). arXiv: [2303.17651](https://arxiv.org/abs/2303.17651) [cs.CL]. URL: <https://arxiv.org/abs/2303.17651>.
- [6] Samuel Schmidgall et al. *Agent Laboratory: Using LLM Agents as Research Assistants*. 2025. DOI: [10.18653/v1/2025.findings-emnlp.320](https://doi.org/10.18653/v1/2025.findings-emnlp.320). arXiv: [2501.04227](https://arxiv.org/abs/2501.04227) [cs.AI]. URL: <https://arxiv.org/abs/2501.04227>.
- [7] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: [2303.11366](https://arxiv.org/abs/2303.11366) [cs.AI]. URL: <https://arxiv.org/abs/2303.11366>.
- [8] Giulio Starace et al. *PaperBench: Evaluating AI's Ability to Replicate AI Research*. 2025. DOI: [10.48550/arXiv.2504.01848](https://doi.org/10.48550/arXiv.2504.01848). arXiv: [2504.01848](https://arxiv.org/abs/2504.01848) [cs.AI]. URL: <https://arxiv.org/abs/2504.01848>.
- [9] Haoyang Su et al. *Many Heads Are Better Than One: Improved Scientific Idea Generation by a LLM-Based Multi-Agent System*. 2024. DOI: [10.18653/v1/2025.acl-long.1368](https://doi.org/10.18653/v1/2025.acl-long.1368). arXiv: [2410.09403](https://arxiv.org/abs/2410.09403) [cs.CL]. URL: <https://arxiv.org/abs/2410.09403>.
- [10] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022. arXiv: [2201.11903](https://arxiv.org/abs/2201.11903) [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.
- [11] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: [2210.03629](https://arxiv.org/abs/2210.03629) [cs.CL]. URL: <https://arxiv.org/abs/2210.03629>.